

Sensor Timing Firmware

The Teensy 4.1 firmware separates physical data acquisition from timing I/O. The Linux host continues acquiring all four cameras, both VLP-16s, and the Xsens MTi-30 continuously. The firmware can qualify a common reference edge, schedule future trigger pulses, observe per-camera exposure and IMU sync edges, and publish timing diagnostics. It neither transports payload data nor proves that a host driver accepted a hardware trigger.

All V6 timing pins are assigned in `firmware_config.h`, but every firmware timing gate is false. On every disabled boot the firmware probes the battery-backed DS3231 and requests `DS3231_OFF`, so a square-wave mode retained from earlier operation is not silently trusted. That shutdown cannot suppress an edge before MCU initialization and cannot be confirmed if the RTC is unreachable; `FIELD_VALID` hardware output gating and commissioning scope tests remain necessary. A build becomes actuating only after the corresponding polarities, line drivers, configuration, and wiring-verification gates are supplied. The scheduler then requires stable reference periods before running, stops on a missing or out-of-range reference, rejects late trigger deadlines, requires camera ExposureActive feedback, and latches faults rather than continuing open-loop. The TPS3897 `FIELD_VALID` input is another required gate for every field-timed path. While it is LOW, firmware suppresses LiDAR reference/NMEA work, drops camera/MTi timing events, keeps scheduled field outputs inactive, and reports a `field_power_invalid` timing fault and discards the relative epoch. U3/U6 OE gates driven by `FIELD_VALID` provide the primary hardware block without adding a component to any signal data path; firmware handling is secondary. U11 eFuse EN/UVLO may remain on down to roughly 4.15 V because of internal hysteresis; it is therefore not the runtime proof that AHCT field logic remains above its 4.5 V guaranteed range. The separate supervisor qualifies the rail near 4.7 V. Relative timing remains unqualified until the assembled board is measured with an oscilloscope or logic analyzer. V6 pulls the DS3231 open-drain `INT/SQW` output to 3.3 V; its falling edge triggers a hardware one-shot. The resulting active-high pulse fans to both VLP-16s, and Teensy D38 captures that same shaped rising edge. The DS3231 datasheet places the 1 Hz square-wave high transition approximately 500 ms after the seconds-register transfer; capturing the falling edge therefore supports association with the next divider boundary, but it is not proof of exact phase. Scope the assembled module SQW against RTC register rollover and the shaped PPS before making any phase claim. A module's 5 V pull-up must never reach the non-5-V-tolerant Teensy. The RTC/one-shot is one shared physical source, not a crosspoint. Enabling it for camera or MTi timing also physically drives both VLP PPS connectors while `FIELD_VALID` is high. `kLidarTimingEnabled` controls LiDAR-specific firmware capture and diagnostics; it is not an independent hardware PPS disconnect. Consequently both VLP PPS branches must be safely wired and electrically commissioned before any common timing function is enabled. The standard Xsens device-time diagnostic remains on `/sensors/imu/sample_time` as `sensor_msgs/TimeReference`. The typed acquisition event used for pairing is published on `/sensors/imu/clock_event` as `xsens_mti_driver/XsensClockEvent`. Each event carries a driver source-instance ID (not a hardware boot ID), OS boot ID, process-start monotonic value, source sequence and sequence width, raw device time, explicit device-clock epoch, ROS-time epoch, and both ROS and monotonic receipt timestamps. Xsens does not expose a hardware boot ID, so the typed event marks that identity invalid and keeps the driver instance separate. Monotonic receipt time is captured in the Xsens callback before the packet enters its queue, not when the queued packet is later processed. The sequence links the event to the receipt-stamped IMU packet. Raw device time establishes chronological order; the finite packet counter is used to report ordinary gaps, while ambiguous wrap/outage gaps are accepted only when raw time advances. A delayed event from a retired process or host boot is rejected.

The canonical `/sensors/imu/data` remains a `sensor_msgs/Imu` stream. The sample-clock adapter also publishes `/sensors/imu/data_timed` as `ig_handle/ClockedImu`, carrying mapped ROS time together with source and receipt clocks, adapter and driver-instance identities, an explicitly unknown hardware boot ID, source, device-clock and ROS epochs, sequence width, uncertainty, and the applied mapping revision. The receipt-offset map is explicitly marked uncalibrated; its uncertainty is unknown until measured. A mapping revision changes whenever an offset is anchored. A source-clock reanchor preserves chronology unless the matched post-request event shows a raw device-clock rollback; that explicit request then establishes a new mapped epoch using the actual new interval. The Xsens driver increments a separate device-clock epoch immediately when raw time rolls back and the packet counter proves forward progression. If the counter is ambiguous, an isolated rollback is skipped. A reset candidate must return both raw time and packet counter below half their prior high-water values, then show increasing raw time and forward counter progression on the next packet. This avoids interpreting a short reordered pair as a reboot. It is still an inference: the device stream exposes no independent boot identifier, so a sufficiently long delayed sequence can resemble a reset. After confirmation, callback-ingress monotonic time rejects packets queued before the new epoch; counter wrap across

the old high-water mark is not used to reject new-epoch data. Events from retired device-clock epochs are rejected. After a ROS time reset, same-host monotonic receipt time rejects events already queued before the reset; the explicit event ROS epoch is used when ROS simulated time is active or the producer runs on another host. DLiO consumes this envelope so clock continuity changes are explicit inputs rather than timestamp guesses.

The Teensy publishes typed `ig_handle/FirmwareTimingEvent` records on `/sensors/timing/firmware_event`. Each record carries its event kind, source index, full 32-bit source sequence, raw relative-MCU time, and relative-clock epoch. The firmware also records the trigger scheduler's command sequence and stamps that command immediately after writing the GPIO output. Each completed camera exposure-feedback event and MTi SyncOut event carries the scheduler sequence that armed it. The sequence relation is scoped to the firmware clock epoch and is not a durable cross-restart capture ID. The trigger timestamp is a software command time, not a measurement of the electrical edge at the camera. The relative-clock epoch is valid only for the current firmware timing session; it is not a boot identifier, UTC, or ROS time. PPS, trigger commands, each camera exposure midpoint, and MTi SyncOut retain distinct source sequences. The older `sensor_msgs/TimeReference` topics remain diagnostic surfaces. Their `header.seq` is not used as event identity because ROS transports may rewrite it.

IG Handle converts the typed firmware records into `ig_handle/AcquisitionTimingEvent` on `/sensors/timing/events`. It records host ROS and monotonic receipt clocks, the host boot ID, firmware build ID when available, source epoch validity, the related trigger source/sequence, sequence gaps, duplicates, reordering, and inferred firmware restarts. The numeric trigger relation is scoped to the firmware clock epoch and remains distinct from the `correlated_capture_id` field, which stays invalid without a restart-safe shared capture identity. A source rollback is treated as a restart only after later ordered records support that interpretation. Since firmware does not expose a durable boot ID, the inferred generation is valid only during this adapter lifetime; `event_id` is scoped by the adapter UUID. The VLP PPS event represents the shared PPS input to the pair, not independent packet-time measurements from both LiDARs.

Each physical VLP launch also records a separate `KIND_LIDAR_CLOUD_HEADER` event for its point-cloud stream. It preserves the cloud's ROS `Header.seq` counter and `Header.stamp`, with the event's ROS and monotonic receipt clocks. The source clock domain is deliberately named `ros_pointcloud_header_unverified`; neither the header stamp nor its sequence is treated as a hardware packet/acquisition clock. The horizontal and vertical VLP records have distinct source IDs. A zero stamp, repeated counter, or out-of-order stamp remains visible in the record, and mapped acquisition time, calibration, and uncertainty remain unavailable. This provides a per-stream diagnostic pair for later characterization without claiming PPS correlation. The event destination is shared with the firmware and camera records and follows the bringup's configured timing-events topic.

Each Spinnaker driver publishes a `CameraFrameTiming` record and an optional `CameraFrameCapture` envelope. The envelope carries the complete image with its camera serial, per-connection stream UUID, full frame counter, raw device timestamp, and SDK-arrival host ROS and monotonic clocks. The timing adapter preserves those values in an acquisition event. This makes the image and its frame identity travel together and lets records be matched by serial, stream UUID, and full counter. It does not establish a relationship between a camera frame and a Teensy exposure-feedback or trigger event. The Teensy counter relates its command and feedback records, but no shared identity currently joins that counter to the Spinnaker frame counter. Establishing that join and measuring the electrical trigger/exposure path still require circuit characterization.

The firmware relative clock begins at the first qualified PPS and is explicitly not UTC. Timing events remain unmapped by default. An operator may provide an offline fit through the adapter's `clock_mappings_file` parameter, but the fit must exactly match source domain, source instance, source epoch, host boot ID, and ROS-time epoch. For a camera this means source domain `spinnaker_device_timestamp_ns`, source instance equal to that camera connection's stream UUID, source epoch unknown, reference domain `ros_time`, reference instance equal to the Linux host boot ID, and reference epoch equal to the adapter's ROS-time epoch. The JSON fit includes a positive mapping revision; increase it whenever the fitted coefficients or paired evidence change.

Create fits only from explicitly identified one-to-one event pairs. The fit utility never pairs by receipt-time proximity and never removes residual outliers. If pair standard uncertainties are unavailable, the output may map timestamps for offline analysis, but the event remains uncalibrated with unknown uncertainty. MARINER uses a camera capture for geometric timing only when the event's serial, stream UUID, full frame counter, raw device timestamp, and both receipt clocks match the image envelope and the mapping is calibrated. It stores the raw and mapped timing provenance with the image. Receipt time alone never becomes acquisition time.

A mapped camera device clock does not itself prove that an image was exposed at a Teensy trigger edge: no shared identity currently joins the Teensy trigger counter to a Spinnaker frame counter. Measuring trigger-to-exposure delay and the electrical path still requires circuit characterization. The adapter is passive and does not enable or change physical timing outputs. The camera itself does not expose a durable hardware boot ID, and sonar sources without hardware acquisition timestamps remain receipt-timed.

GRANDE's `runtime`, `raw`, and `slam` bag profiles include the typed timing events, timed IMU and Xsens clock-event topics, and configured sensor topics. The raw profile was composed offline and includes both VLP point streams. The compact `navigation` and `oracle` profiles omit acquisition-timing topics; use a runtime, raw, or SLAM recording when timing provenance is needed.

The currently available square wave is not a verified UTC/GNSS phase source. Published firmware time references use a monotonic epoch beginning at the first qualified edge and explicitly do not claim UTC. On the existing `sensor_msgs/TimeReference` diagnostics, `header.stamp` is only the ROS publication-receipt time and `time_ref` is the unmapped relative MCU epoch. These topics are non-authoritative and are not a ROS-time calibration contract.

LiDAR product identity and timing links

The cloud-header observer preserves the original serialized product identity in `AcquisitionTimingEvent.correlated_capture_ids`. A derived clock revision keeps that identity. MARINER archives the cloud and event separately and records their relationship with the original mapper context, timing-source lifetime and clock revision. An exact, unique product with matching frame, sequence, timestamp and context can establish a software identity link; it does not measure PPS phase, transport delay, acquisition-clock calibration or physical synchronization.

A filtered cloud can retain a declared source-topic/header relationship without proving its parent payload identity. It remains unresolved. Repeated identical products in different odometry generations make the relationship ambiguous; an append-only revision supersedes the earlier unique claim. All candidate identities remain available after cache eviction and checkpoint reopening. Receipt time never substitutes for missing acquisition time, and these records cannot authorize navigation or outputs.

Electrical boundary

The installed-camera inventory must be checked against the physical labels. The current Teledyne reference names the official color model `FG-PGE-50S5C-IP`; the repository's `FG-PGE-50S5C-C-IP` spelling is not assumed to be a second electrical variant.

For the current Forge IP67 family, the M8 eight-pin GPIO interface assigns pin 1 to 8-24 V camera input power, pin 2/Line0 to the opto-isolated input, pin 3 to opto ground, pin 6/Line1 to the opto-isolated output, and pin 7 to camera power ground. Published typical thresholds are 1.55 V low and 2.6 V high for Line0; Line1 is specified as approximately 0.56 V low and 5 V high under the vendor's measurement conditions. The Teensy accepts only 0-3.3 V inputs and its normal output recommendation is 4 mA. Therefore neither Forge line connects directly to the MCU. The board must provide characterized, 3.3 V-safe buffered inputs and outputs and defined returns. OPTOOUT is open collector, so each camera gets a separate starting-value 1.0 kOhm pull-up to board 3.3 V referenced to OPTOGND; cable rise time and sink current must be measured. V6 carries only M8 pins 2, 3, and 6 for each camera, does not carry camera-power ground pin 7, and does not power the PoE cameras. Touching 1x6 J5 carries cameras 1-2 and 1x6 J7 carries cameras 3-4; each camera's OPTOGND is bonded to board signal ground. Use IP-rated M8 cables or sealing plugs to retain the camera's ingress rating.

V6 uses one D34 command and four hardware-buffered fanout branches; it does not claim four sequential GPIO writes are simultaneous. R66-R69 are separate 10 kOhm connector-side pulldowns that hold `CAM1_OPT0IN` through `CAM4_OPT0IN` low during MCU reset/high-Z, core-off, driver-disable, or an open harness. They are static fail-low loads, not RC timing filters. Characterize branch skew on the assembled board. Configure each camera separately for `FrameStart`, the selected Line0 rising edge, and Line1 `ExposureActive`; both device-side functions have separate false-by-default firmware gates. Verify every accepted trigger and exposure edge on the assembled harness before enabling the firmware gate. The current host profile remains free-running and unchanged.

The VLP-16s acquire continuously. Their input is a clock/phase reference, not a scan trigger. The official manual requires a PPS high above 3.0 V and below 5.0 V, low below 1.2 V, and at least 2 mA source current. NMEA and PPS must alternate; PPS-to-NMEA separation is at least 50 ms and the NMEA sentence must finish at least 300 ms before the next PPS. The current firmware has no verified UTC/GNSS source. Its optional nonblocking Serial1 path emits a checksum-correct status-V GPRMC sentence from the local RTC only after the wiring, polarity, RTC state, and measured register-rollover-to-SQW-to-PPS phase are commissioned; it makes no GNSS-position or UTC-traceability claim. The status remains **V**, with blank position fields; firmware must never fabricate receiver-valid status **A**. On each VLP, read back and save both the PPS Qualifier and GPS Qualifier **Require GPS Receiver Valid** settings as OFF before enabling the RTC-only stream. The manual shows OFF as a default, but an assumed default is not commissioning evidence. Verify the delayed PPS Locked state, packet PPS status 2, and copied RMC/time continuity independently on both units. The Teensy UART peripheral, not an extra inverter IC, owns the VLP-required TX polarity inversion; the board's AHCT fanout is non-inverting. Firmware starts a sentence 100 ms after the shaped PPS leading edge. With the nominal 10 ms one-shot, this preserves about 90 ms from PPS trailing edge to NMEA start instead of relying on the 50 ms minimum. It suppresses the sentence if the main loop misses the 200 ms start deadline, aborts a pending tail after the 500 ms enqueue deadline, and feeds only currently available UART-buffer capacity. A conservative 80-byte, 9600-baud bound is 83,334 us, so 500,000 + 83,334 us still leaves the manual's final 300 ms quiet window even at the firmware's minimum accepted 900 ms reference period. The sentence contains the next PPS second, including calendar rollover, because Rev-F describes the LiDAR applying the received time at that next leading edge. Scope verification is mandatory because enabling a second hardware inversion would silently restore the wrong polarity. Dual-VLP phase locking additionally requires a locked PPS, an RPM multiple of 60 between 300 and 1200, and distinct measured phase offsets configured and saved on the sensors. The one-shot and common electrical fanout, not two MCU writes, create the two LiDAR PPS branches.

Live USB and udev inspection on 2026-08-26 confirmed the installed device identifies as an Xsens MTi-30 AHRS. Its serial identity remains host inventory, not an electrical interface requirement. The existing USB connection continues to carry IMU data; V6 adds only the separate SyncIn, SyncOut, and signal-return timing connector. R65 is a 10 kOhm connector-side pulldown that holds **MTI_SYNCIN** low during MCU reset/high-Z, core-off, driver-disable, or an open harness. The encased MTi-30 synchronization contract accepts 0-0.8 V low and 2.5-20 V high on SyncIn/ClockSync; SyncOut is 0-0.4 V low and above 2.9 V high. Firmware cannot determine the selected MTi **SyncSettings**. A 10 kOhm receiver-side pulldown makes an open SyncOut cable deterministic low before the board's inverting Schmitt receiver, so the MCU input idles high rather than floating. The optional MCU output is only a time-event marker for the continuously sampled IMU; it must not select **StartSampling** or reduce navigation data to the camera's current 5 Hz cadence. It therefore remains disabled until polarity, voltage, cable pin, the SyncIn event, the SyncOut feedback event, and a corresponding packet marker are independently verified in MT Manager.

The Imagenex DT100 and Ping360 are intentionally not connected to V6 in this revision. That is a safety boundary, not a statement that hardware sync is impossible. The DT100 has Sync IN/OUT circuitry, but a vendor report identified damage after power was applied to those lines. Do not drive either DT100 sync pin until the exact connector, direction, polarity, voltage levels, and common-reference requirements are vendor-verified. The current safe contract uses host/driver timestamps for both sonars and fuses their asynchronous measurements against a continuous-time trajectory; it does not label them hardware-synchronized.

Firmware structure and validation

sensor_sync.h owns the side-effect-free state machine, bounded edge mailboxes, and continuous rollover-safe relative-epoch mapper. GPIO ISRs only deposit timestamped edges; the timer ISR is the sole owner of scheduler, feedback, fault, and relative-time state. Mailbox overflow latches synchronization off. **sensor_sync_runtime.h** owns Teensy GPIO, camera exposure midpoint capture, IMU events, feedback deadlines, and safe inactive outputs. **firmware_pin_contract.h** rejects cross-owner aliases, RTC-I2C pin collisions, unsupported telescope PWM assignments, and invalid digital pins before either runtime initializes. **main.ino** owns ROS publication, RTC edge labelling, the bounded nonblocking PPS-to-NMEA state machine, the timer ISR, and the disabled telescope runtime.

The pure scheduler is suitable for non-actuating host compilation and state transition checks. These checks are performed from an ephemeral harness rather than a repository validator. They check logic only. A valid Teensy build requires a Teensyduino core, roserial headers, and RTCLib. Acceptance additionally requires a current-limited bench supply, no motor or sensor

payloads connected for initial bring-up, safe inactive outputs during boot/reset/disconnect, instrumented edge-to-edge skew and jitter, deliberate missing-reference and feedback-fault injection, MTi packet-marker verification, VLP PPS-lock/phase-lock status, and read-back of both VLP qualifier pairs with Receiver Valid requirements disabled, packet PPS status 2, copied status-V RMC/time continuity, and camera exposure/frame-counter correlation. Passing simulation or host tests is not physical synchronization evidence.

The telescope motor driver also has external hard pulldowns and a common default-low enable so reset, boot, disconnected MCU, or high-impedance GPIO cannot energize either half-bridge. The independent hard E-stop remains in the 12 V motor-power path. Firmware fail-closed states supplement those electrical controls; they do not replace them.

Structured acquisition records and characterization

AcquisitionTimingEvent carries source event identity, acquisition session, boot/clock epoch validity, original 64-bit sequence/counter, raw clock domain/time, receipt ROS and monotonic clocks, host boot identity, mapping revision, calibrated state, known/unknown uncertainty, and capture correlation. Xsens ClockedImu retains its device sample identity alongside the mapped standard IMU message. The camera driver publishes CameraFrameCapture with its actual frame counter and device timestamp. Diagnostic text is not the identity authority.

Each camera retains independent frame, trigger, and exposure identities. The adapter does not currently establish a hardware correspondence between a Teensy trigger counter and a Spinnaker frame counter. Such a correspondence must come from commissioned device readback and measured events; sharing an electrical trigger does not merge camera counters or prove the join. Duplicate, reordered, dropped, ambiguous, or counter-width-invalid events remain explicit. Unknown ordering preserves the established source generation rather than reusing generation zero.

Clock offset/rate fitting uses identified event correspondences. Receipt delay alone cannot establish phase. Raw clocks remain reconstructable. A clock mapping that breaks chronology starts a declared mapped continuity epoch; timestamps are never clamped or assigned fabricated intervals. DS3231 time remains local. PPS availability does not make it UTC. Both VLP packet/point timing branches remain independent; gps_time=false is not silently changed.

For the separately authorized circuit characterization:

1. Record original firmware edge events, camera frame counters/device clocks, ClockedImu/sample markers, both VLP timing records, and standard sensor data. Use the runtime profile for the odometry baseline and the raw profile for timing-only capture, with the required camera transport.
2. Measure PPS phase, each trigger-to-exposure delay, four independent exposure feedback branches, and the IMU SyncIn/SyncOut-to-sample relationship.
3. Save instrument timebase, wiring/polarity, source identities, device readback, uncertainty, dropped-event counts, and calibration revision with the recording.
4. Fit clock offset/rate only for uniquely identified pairs. Retain unresolved and reset intervals separately, then check both LiDAR packet/point conventions.
5. Compare DLiO with the fixed-alignment mocap baseline after timing is measured.

Host fixtures cover association, drops, duplicate/reordered data, reset identity, clock mapping revisions, failed reload rollback, and late correlation. They do not measure electrical phase or enable firmware outputs. The vertical LiDAR's reported hardware fault remains unresolved; its software/fixture support stays available. Sonar hardware acquisition timing and acoustic calibration remain unmeasured.

The reviewed timing-only staged source compiled for Teensy 4.1 on 2026-09-28 with the pinned toolchain and generated FirmwareTimingEvent header. No upload or output enablement occurred. Historical firmware metadata and the separate uncommitted lighting image do not certify this timing-only source.

Primary references

- Teledyne FLIR, [Forge 5 MP GigE camera family](#), model and acquisition-feature reference.
- Teledyne FLIR, [Forge IP67 GPIO electrical characteristics](#), M8 pinout, thresholds, delays, and line ratings.

- Teledyne FLIR, [Forge IP67 acquisition control](#), FrameStart trigger-source/mode/overlap contract.
- Ouster (Velodyne), [VLP-16 User Manual, 63-9243 Rev. F](#), PPS/NMEA electrical, timing, and phase-lock requirements.
- Analog Devices, [DS3231 datasheet](#), open-drain `INT/SQW` electrical contract.
- Xsens, [MTi 10-series and MTi 100-series User Manual, MT0605P](#), encased MTi connector, SyncIn/SyncOut, and ClockSync electrical characteristics.
- PJRC, [Teensy 4.1 technical specifications](#), MCU GPIO voltage and current limits.

M1 baseline preparation on IGHandle

Aaron owns the ROS 2 port on the separate NUC. IGHandle's deployed ROS 1 acquisition and native mapping remain the baseline for timing and mocap work. This work does not modify the NUC checkout, Heron startup, or firmware output enablement. Keep the failed vertical LiDAR powered off. Select and qualify its replacement; repair the failed unit only if practical as a backup.

The installed timing adapter and mocap bridge share `sensors.clock_mapping`. The helper parses and applies measured affine mappings while preserving raw clocks and unknown uncertainty; it does not derive phase from receipt delay. The existing `fit_clock_mapping.py` operates offline on uniquely identified event pairs. Fit each source/device epoch independently and retain unassociated or ambiguous events rather than fabricating pair identity.

For each authorized measurement, preserve instrument timebase/uncertainty, source boot/session, original event IDs, device readback, frame, wiring revision and clock-mapping revision. Camera branches require their own measured trigger-to-exposure relation. The functioning LiDAR branch is characterized first; retain support for the second branch but do not power the faulty unit. The DS3231 epoch remains local, and existing VLP `gps_time=false` is unchanged.

Prepare a recording only when explicitly requested. Use the `runtime` profile for the mocap/odometry baseline; the `raw` profile retains acquisition data but does not include local odometry. The resolved baseline topic list must include original firmware edges, acquisition timing, camera timing/captures, clocked IMU/sample time, LiDAR points and available packets, mocap timing/status, local odometry, DLiO accepted-state continuity, and transforms. Missing optional channels remain documented limitations. A recording cannot enable triggers, acoustic transmission or drive.

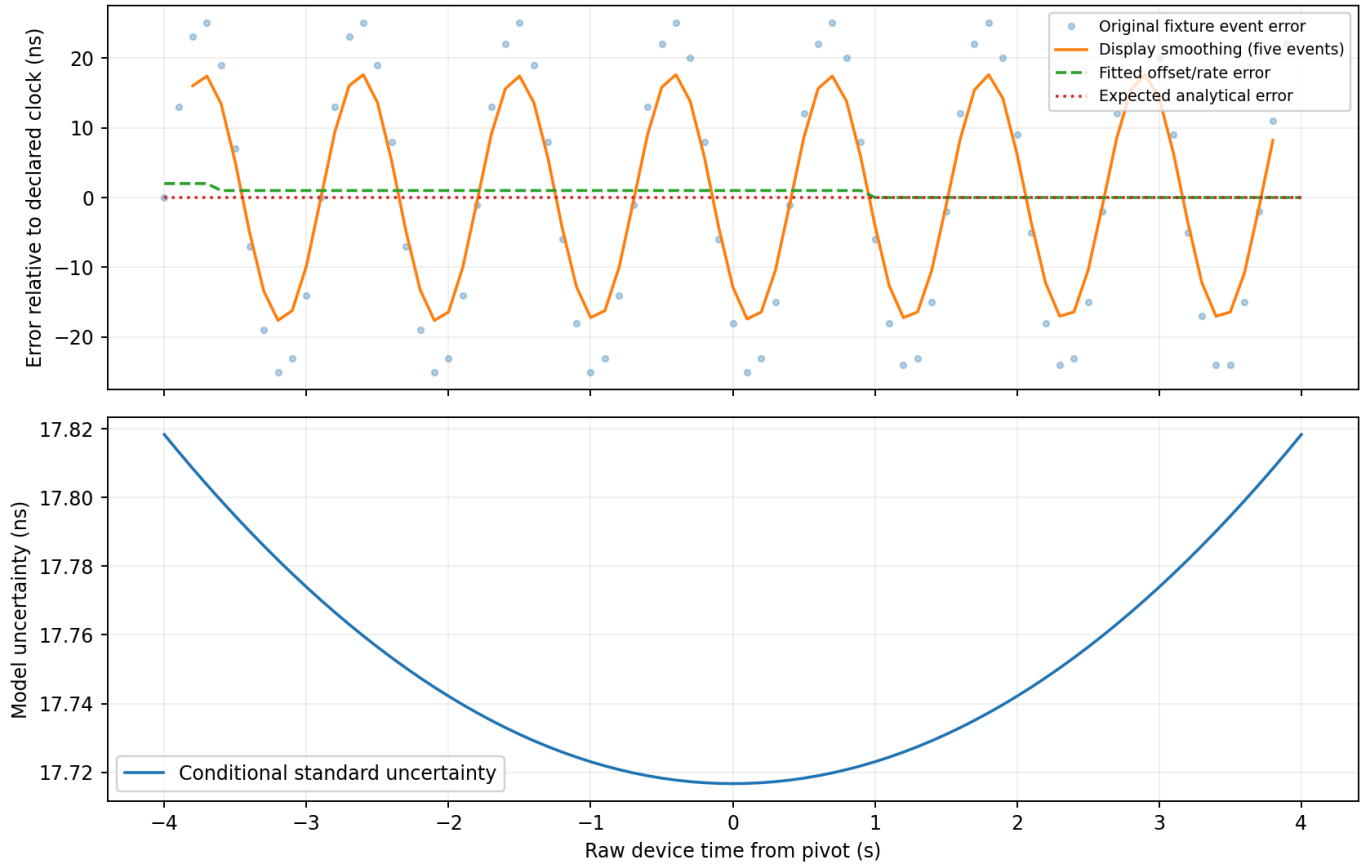
After recording closes, fit identified clock pairs with the existing offline tool and compare odometry using `compare_mocap_odometry.py`. Use one declared initialization interval and score later motion, with no fitted trajectory shift or repeated alignment. Qualified bag/CSV fixtures cover duplicate products, tracking gaps, resets, clock revisions, producer identity, frame mismatch and unresolved events; they measure software behavior only. Physical phase, camera calibration, mocap accuracy and DLiO drift remain unmeasured.

The installed offline fitter accepts identified event-pair CSV records:

```
roslaunch ig_handle fit_clock_mapping.py IDENTIFIED_PAIRS.csv FIT.json \
  --revision POSITIVE_REVISION
```

Use `--help` for the required pair identity, clock scope and uncertainty fields. A fitted file is an analysis result until its measured correspondence and uncertainty are reviewed; it never enables timing outputs automatically.

SOFTWARE FIXTURE — affine clock model Synthetic event pairs; no physical phase or electrical timing claim



Software fixture, 2026-09-29. Synthetic identified event pairs test the installed fitter and shared mapping helper at a declared 20 ppm rate and 2 ms pivot offset. Original event error, display-only smoothing, fitted mapping and analytical expectation are separate. Large absolute clock values remain integer nanoseconds. Conditional uncertainty uses the stated fixture pair uncertainty; unknown physical uncertainty remains unknown. This is not electrical timing data.